

F O R M A C I Ó N P R Á C T I C A

MÓDULO 11

Seguridad, privacidad y control de calidad del código generado

OpenAI Codex — Curso Completo
(Módulo obligatorio)

Módulo crítico de seguridad

Este módulo es obligatorio. Todo código generado por IA debe pasar por los controles de seguridad y calidad aquí descritos antes de llegar a producción.

Versión 1.0 — Febrero 2026

1. Objetivos del módulo

Al finalizar este módulo, el alumno será capaz de:

- **Aplicar prácticas de seguridad operativa** al trabajar con Codex: gestión de secretos, revisión de dependencias, prevención de prompt injection y mitigación de riesgos en la cadena de suministro de software (supply chain).
- **Comprender los controles de retención de datos** a nivel organización que ofrece la plataforma OpenAI, incluyendo Zero Data Retention (ZDR) y políticas de privacidad empresarial.
- **Establecer un «Definition of Done» de calidad** para código generado por IA, integrando quality gates con tests, linters, scanners de dependencias y cobertura mínima.
- **Realizar auditorías de seguridad** sobre código generado y producir informes con hallazgos, correcciones y tests de regresión.

2. Teoría

2.1 Seguridad operativa de Codex

OpenAI Codex opera dentro de un modelo de seguridad por capas diseñado para minimizar riesgos. Este modelo se estructura en dos niveles que trabajan de forma complementaria:

Capa de seguridad	Descripción	Valor por defecto
Sandbox (entorno aislado)	Limita lo que Codex puede hacer técnicamente: acceso al sistema de archivos, red y ejecución de comandos.	Solo escritura en workspace actual; red deshabilitada
Política de aprobación	Define cuándo Codex debe detenerse y pedir autorización al usuario antes de ejecutar una acción.	Aprobación requerida para acciones fuera del workspace

Modos de sandbox disponibles:

Modo	Permisos	Caso de uso
read-only	Solo lectura de archivos	Revisión y análisis de código
workspace-write	Lectura/escritura en workspace; sin red	Desarrollo estándar (recomendado)

<code>danger-full-access</code>	Acceso completo a sistema, archivos y red	Solo entornos controlados/CI
---------------------------------	---	------------------------------

⚠ Principio fundamental

La red está deshabilitada por defecto. Codex no puede realizar conexiones salientes a menos que se habilite explícitamente con `network_access = true` en la configuración del sandbox.

Políticas de aprobación:

- `untrusted`: aprobación requerida para toda acción (más restrictivo).
- `on-request`: aprobación solo para acciones que excedan el sandbox.
- `on-failure`: aprobación solo si un comando falla.
- `never`: sin aprobaciones (solo para entornos muy controlados).

Entornos de ejecución:

- **Codex Cloud**: Contenedores aislados gestionados por OpenAI. El código nunca toca la máquina del desarrollador.
- **Codex CLI / Extensión IDE**: Sandbox a nivel de sistema operativo (seccomp/Landlock en Linux, Seatbelt en macOS). Permisos de escritura limitados al workspace activo.

2.2 Prompt injection y web search

Codex implementa una búsqueda web con caché por defecto para reducir la exposición a prompt injection. En lugar de navegar páginas en vivo, utiliza un índice de resultados pre-indexados mantenido por OpenAI.

Modo de búsqueda	Comportamiento	Riesgo de injection
<code>cached</code>	Resultados pre-indexados (defecto)	Bajo
<code>live</code>	Navegación en vivo de páginas	Alto
<code>disabled</code>	Búsqueda web desactivada	Nulo

🔥 Riesgo de prompt injection

Cuando se habilita búsqueda en vivo (`web_search = "live"`) o acceso a red, el contenido externo puede contener instrucciones maliciosas diseñadas para manipular el comportamiento del agente. Trata siempre los resultados web como no confiables.

2.3 Controles empresariales de datos

OpenAI ofrece diferentes niveles de control sobre la retención de datos según el tipo de suscripción y acuerdo empresarial:

Control	API estándar	Enterprise/Edu	ZDR (aprobado)
Retención de datos	30 días	Configurable por admin	Sin retención
Entrenamiento con datos	No por defecto	No por defecto	No
Revisión humana	Sí (abuso)	Limitada	Deshabilitada
Cifrado	AES-256 + TLS 1.2+	AES-256 + TLS + EKM	AES-256 + TLS + EKM
Residencia de datos	No	Sí (US, EU, UK, JP...)	Sí

Zero Data Retention (ZDR): Es una opción para clientes API que requieren que los inputs y outputs no se almacenen más allá del procesamiento inmediato. Requiere aprobación previa de OpenAI y la aceptación de requisitos adicionales. Cuando ZDR está habilitado, el parámetro `store` siempre se trata como `false`.

Nota importante sobre retención

En junio de 2025, una orden judicial en EE.UU. obligó a OpenAI a preservar datos de usuarios temporalmente. La orden finalizó el 26 de septiembre de 2025 y se restauraron las políticas estándar. Los clientes con acuerdos ZDR no fueron afectados.

2.4 Principios fundamentales de seguridad con IA generativa

Principio 1: NUNCA pegar secretos en prompts

Las API keys, tokens de acceso, contraseñas y cualquier credencial jamás deben incluirse en los prompts enviados a Codex ni a ningún modelo de lenguaje. Incluso con ZDR, existe riesgo de exposición accidental en logs, historial de conversaciones o trazas de depuración.

```
# ✗ MAL: Secreto en el prompt
codex "Conecta a la BD con password=S3cr3t!K3y"

# ✓ BIEN: Variables de entorno
codex "Conecta a la BD usando la variable de entorno $DB_PASSWORD"
```

Principio 2: Revisión de licencias y dependencias

Todo paquete sugerido por Codex debe verificarse antes de añadirlo al proyecto. La revisión debe incluir: licencia compatible con el proyecto, estado de mantenimiento (fecha del último

commit, issues abiertos), vulnerabilidades conocidas (CVEs) y popularidad/confiabilidad de la comunidad.

Principio 3: Análisis estático de seguridad (SAST) y linters

Todo código generado debe pasar por herramientas de análisis estático antes de integrarse. Herramientas recomendadas según stack:

Lenguaje	Linter / SAST	Scanner de dependencias
Python	bandit, pylint, ruff	pip-audit, safety
JavaScript/TS	ESLint (security plugin), semgrep	npm audit, snyk
Java	SpotBugs, PMD, SonarQube	OWASP Dependency-Check
Go	gosec, staticcheck	govulncheck
General	semgrep, CodeQL	Trivy, Gype

Principio 4: Validación de entradas

Nunca confiar en datos proporcionados por el usuario. Todo input debe sanitizarse y validarse tanto en el cliente como en el servidor. Codex puede generar código que no incluye validación adecuada por defecto.

Principio 5: Autorización (AuthZ) y autenticación (AuthN)

Verificar que el código generado implementa correctamente los controles de acceso. Codex puede generar endpoints funcionales pero sin middleware de autenticación o comprobaciones de permisos.

Principio 6: Logging sin PII ni secretos

Los logs de aplicación deben registrar eventos de seguridad relevantes, pero nunca datos personales identificables (PII), tokens, claves API ni información sensible del usuario.

3. Buenas prácticas

3.1 Checklist de seguridad para tareas Codex

Antes de aceptar cualquier código generado por Codex, aplica esta lista de verificación:

☐	Pregunta de verificación	Acción requerida
☐	¿El código toca autenticación o autorización?	Revisar middleware, tokens, roles y permisos
☐	¿Añadió nuevas dependencias?	Verificar CVEs, licencia y mantenimiento
☐	¿Maneja inputs del usuario?	Validar sanitización y escape de datos
☐	¿Maneja errores correctamente?	Verificar try/catch, códigos HTTP, mensajes genéricos
☐	¿Los logs exponen PII o secretos?	Auditar console.log, logger.info, print()
☐	¿Accede a bases de datos?	Revisar queries parametrizadas vs concatenación
☐	¿Expone endpoints nuevos?	Verificar rate limiting, CORS, validación
☐	¿Incluye hardcoded secrets?	Buscar strings, tokens, URLs con credenciales

3.2 Quality gates: Definition of Done para código IA

Un «Definition of Done» (DoD) establece los criterios mínimos que debe cumplir cualquier código generado por IA antes de ser aceptado en el proyecto:

Quality gate	Criterio mínimo	Herramientas ejemplo
Tests unitarios	Cobertura $\geq$ 80% en código nuevo	pytest, jest, JUnit
Linters	0 errores, 0 warnings críticos	ESLint, ruff, golangci-lint
SAST	0 hallazgos de severidad alta/crítica	bandit, semgrep, CodeQL
Scan dependencias	0 CVEs críticas/altas sin mitigar	npm audit, pip-audit, Trivy

Revisión de licencias	Compatible con licencia del proyecto	license-checker, FOSSA
Revisión humana	Al menos 1 revisor aprueba el PR	GitHub PR review, GitLab MR

✓ Regla de oro

Ningún código generado por IA debería tener menos escrutinio que el código escrito por humanos. En la práctica, debería tener más, porque la IA puede generar código funcional pero inseguro sin que el desarrollador lo note a simple vista.

3.3 Configuración de seguridad recomendada para Codex

Configuración recomendada para el archivo ~/.codex/config.toml en entornos de desarrollo:

```
# ~/.codex/config.toml – Configuración segura recomendada

# Modelo por defecto
model = "gpt-5-codex"

# Política de aprobación: pedir autorización para acciones externas
approval_policy = "on-request"

# Sandbox: escritura limitada al workspace
sandbox_mode = "workspace-write"

# Red deshabilitada por defecto
[sandbox_workspace_write]
network_access = false

# Búsqueda web en modo caché (menor riesgo de injection)
web_search = "cached"

# Excluir variables sensibles del shell del agente
[shell_environment_policy]
ignore_default_excludes = false
```

4. Errores comunes

4.1 Copiar trazas con tokens o API keys

Uno de los errores más frecuentes y peligrosos es pegar mensajes de error, stack traces o logs directamente en el prompt de Codex sin sanitizar. Estas trazas pueden contener tokens de sesión, API keys, cadenas de conexión a bases de datos u otra información sensible.

✗ Error típico

Copiar un error de producción como "ConnectionError: Failed to connect to db.prod.company.com:5432 with user=admin password=Pr0d_S3cr3t" directamente en el prompt de Codex. El secreto quedará en los logs de la plataforma durante al menos 30 días.

Cómo prevenirlo:

- Sanitiza SIEMPRE las trazas antes de pegarlas: reemplaza credenciales por placeholders como [REDACTED] o <PASSWORD>.
- Usa variables de entorno en lugar de valores literales en los mensajes de error.
- Configura tu framework de logging para enmascarar campos sensibles automáticamente.
- Si ya pegaste un secreto, rotalo inmediatamente: genera nuevas credenciales y revoca las anteriores.

4.2 Aceptar dependencias sugeridas sin revisar

Codex puede sugerir bibliotecas que parecen adecuadas pero que presentan riesgos. Los principales problemas incluyen:

- **Vulnerabilidades conocidas (CVEs):** Paquetes con versiones desactualizadas que contienen fallos de seguridad documentados.
- **Licencias incompatibles:** Una dependencia con licencia GPL en un proyecto propietario puede obligar a liberar el código fuente.
- **Paquetes abandonados:** Sin mantenimiento activo, no recibirán parches de seguridad futuros.
- **Typosquatting:** Paquetes con nombres similares a bibliotecas populares pero que contienen código malicioso (ejemplo: `crypt0graphy` en lugar de `cryptography`).

Verificación obligatoria antes de instalar:

```
# Python: verificar vulnerabilidades
pip-audit

# Node.js: auditoría de seguridad
```



```
npm audit

# Verificar licencias (Node.js)
npx license-checker --summary

# Scanner general con Trivy
trivy fs --scanners vuln,license .
```

4.3 Otros errores frecuentes

- **Desactivar el sandbox sin justificación:** Usar `--dangerously-bypass-approvals-and-sandbox` o `sandbox_mode = "danger-full-access"` fuera de entornos aislados compromete la seguridad del sistema.
- **No validar inputs:** Aceptar código generado que procesa entrada del usuario sin sanitización (SQL injection, XSS, path traversal).
- **Exponer errores internos al cliente:** Devolver stack traces completos en respuestas HTTP en lugar de mensajes genéricos.
- **Omitir rate limiting:** Crear endpoints públicos sin límites de peticiones, permitiendo ataques de denegación de servicio.

5. Casos reales

5.1 Caso 1: Hardening de endpoints

Escenario: Un equipo genera una API REST con Codex. Los endpoints funcionan correctamente pero carecen de controles de seguridad básicos.

Problemas detectados:

- Endpoint POST /api/users sin autenticación: cualquier persona puede crear usuarios.
- Sin rate limiting: vulnerable a ataques de fuerza bruta.
- CORS configurado como * (permite peticiones desde cualquier origen).
- Sin validación de Content-Type: acepta cualquier formato de datos.

Solución aplicada con Codex:

```
# Prompt efectivo para hardening:
"Revisa el endpoint POST /api/users y aplica las siguientes
mejoras de seguridad:
1. Añade middleware de autenticación JWT
2. Implementa rate limiting (máx 10 req/min por IP)
3. Restringe CORS a https://midominio.com
4. Valida Content-Type: application/json
5. Sanitiza todos los campos del body
6. Añade tests para cada control de seguridad"
```

5.2 Caso 2: Sanitización de logs y protección de PII

Escenario: Un microservicio de pagos registra transacciones en logs para debugging. Al auditar los logs en producción se descubre que contienen números de tarjeta de crédito, emails y nombres completos de los clientes.

Antes (inseguro):

```
// ✗ Log con PII expuesta
logger.info("Pago procesado", {
  user: "Juan Pérez",
  email: "juan@empresa.com",
  card: "4111-1111-1111-1111",
  amount: 150.00
});
```

Después (seguro):

```
// ☑ Log sanitizado
```

```
logger.info("Pago procesado", {
  userId: "usr_a1b2c3",          // ID opaco, no nombre
  emailHash: sha256(email),      // Hash, no email real
  cardLast4: "****_****_****-1111",
  amount: 150.00,
  transactionId: "txn_x7y8z9"
});
```

Prompt para Codex:

```
"Revisa todos los llamados a logger.* en el directorio src/
e identifica cualquier log que exponga PII (nombres, emails,
números de tarjeta, teléfonos, direcciones IP de usuarios).
Reemplaza los datos sensibles con versiones enmascaradas.
Añade tests que verifiquen que ningún log contiene patrones
de PII (regex para emails, tarjetas, etc.)."
```

6. Laboratorio (L11) — Auditoría de seguridad y hardening

🔗 Objetivo del laboratorio

Detectar y corregir vulnerabilidades típicas en un servicio pequeño utilizando Codex como asistente de seguridad, generando un informe de hallazgos, parches correctivos y tests de regresión.

6.1 Requisitos previos

- OpenAI Codex CLI instalado y configurado.
- Node.js 18+ y npm (o Python 3.10+ con pip, según el stack elegido).
- Git instalado y configurado.
- Editor de código con extensión de Codex (recomendado).

6.2 Preparación del entorno

Paso 1: Crear el repositorio del laboratorio y la aplicación vulnerable.

```
mkdir codex-lab11 && cd codex-lab11
git init
npm init -y
npm install express
```

Paso 2: Crear el servicio vulnerable (`server.js`). Este archivo contiene vulnerabilidades intencionadas para el ejercicio:

```
const express = require("express");
const app = express();
app.use(express.json());

// Base de datos simulada
const users = [];
const ADMIN_TOKEN = "SuperSecretToken123!"; // ✗ Hardcoded secret

// ✗ Vuln 1: Sin autenticación
app.post("/api/users", (req, res) => {
  const { name, email, password } = req.body;
  // ✗ Vuln 2: Sin validación de inputs
  users.push({ name, email, password }); // ✗ Almacena password en texto plano
  // ✗ Vuln 3: Log con PII
  console.log(`Nuevo usuario creado: ${name}, ${email}, ${password}`);
  res.json({ message: "Usuario creado", user: { name, email, password } });
  // ✗ Vuln 4: Devuelve password en la respuesta
```

```
});

// ✗ Vuln 5: Inyección SQL simulada (concatenación de strings)
app.get("/api/users/search", (req, res) => {
  const { q } = req.query;
  // Simulación de query insegura
  const results = users.filter(u =>
    u.name.includes(q) || u.email.includes(q)
  );
  res.json(results); // ✗ Devuelve todos los campos incluyendo passwords
});

// ✗ Vuln 6: Endpoint admin sin authz
app.delete("/api/users/:id", (req, res) => {
  users.splice(req.params.id, 1);
  res.json({ message: "Usuario eliminado" });
});

// ✗ Vuln 7: Sin CORS, sin rate limiting, sin helmet
app.listen(3000, () => {
  console.log(`Server en puerto 3000, token admin: ${ADMIN_TOKEN}`);
  // ✗ Vuln 8: Log expone secreto al iniciar
});
```

Paso 3: Hacer commit del código vulnerable.

```
git add .
git commit -m "feat: servicio inicial con vulnerabilidades controladas (lab11)"
```

6.3 Ejecución del laboratorio

Tarea A — Revisión de seguridad con Codex

Pide a Codex que realice una auditoría completa del servicio:

```
codex
> "Realiza una auditoría de seguridad completa de server.js.
> Identifica todas las vulnerabilidades, clasifícalas por
> severidad (crítica/alta/media/baja) y genera un informe
> en formato Markdown con:
> 1. Tabla de hallazgos (ID, severidad, descripción, línea)
> 2. Recomendación de fix para cada hallazgo
> 3. Referencias a CWE/OWASP cuando aplique"
```

Resultado esperado: Codex debería identificar al menos las 8 vulnerabilidades intencionadas y generar un informe SECURITY_AUDIT.md.

Tarea B — Aplicar parches de seguridad

Pide a Codex que corrija las vulnerabilidades:

```
> "Aplica los siguientes parches de seguridad en server.js:  
> 1. Añade autenticación JWT con middleware  
> 2. Valida y sanitiza todos los inputs con express-validator  
> 3. Hashea passwords con bcrypt antes de almacenar  
> 4. Elimina PII de todos los logs  
> 5. Mueve secretos a variables de entorno (.env)  
> 6. Añade helmet, cors restringido y rate limiting  
> 7. No devuelvas nunca el password en respuestas  
> 8. Añade authz al endpoint DELETE (solo admin)"
```

Tarea C — Tests de regresión

Genera tests que verifiquen que las vulnerabilidades están corregidas:

```
> "Genera tests con Jest y supertest que verifiquen:  
> 1. POST /api/users sin token devuelve 401  
> 2. POST /api/users con datos inválidos devuelve 400  
> 3. GET /api/users/search no devuelve passwords  
> 4. DELETE /api/users/:id sin rol admin devuelve 403  
> 5. Los logs NO contienen emails ni passwords (mock logger)  
> 6. Rate limiting bloquea tras 10 requests/min"
```

6.4 Validación con herramientas externas

Ejecuta las siguientes validaciones para confirmar la efectividad de los parches:

```
# Ejecutar tests  
npx jest --coverage  
  
# Auditoría de dependencias  
npm audit  
  
# Linter de seguridad (si se instaló eslint-plugin-security)  
npx eslint --ext .js src/  
  
# Scanner general (requiere Trivy instalado)  
trivy fs --scanners vuln .
```

6.5 Resultado esperado

Al finalizar el laboratorio, el repositorio debe contener:

Archivo	Contenido
SECURITY_AUDIT.md	Informe de auditoría con tabla de hallazgos y severidad

server.js	Servicio parcheado con todos los controles de seguridad
.env	Variables de entorno con secretos (NO commitar)
.gitignore	Incluye .env y node_modules
tests/security.test.js	Tests de regresión para cada vulnerabilidad corregida
package.json	Dependencias actualizadas y scripts de test

6.6 Limpieza

Limpieza obligatoria

Al finalizar el laboratorio, elimina el repositorio y cualquier reporte que contenga datos sensibles del entorno de pruebas.

```
# Eliminar el repositorio del laboratorio
cd ..
rm -rf codex-lab11/

# Si generaste reportes fuera del repo, elimínalos también
rm -f ~/security-audit-lab11.md

# Verifica que no quedan archivos .env en otros directorios
find ~ -name ".env" -path "*/lab11/*" -delete
```

7. Resumen y puntos clave

Concepto	Punto clave
Sandbox de Codex	Red deshabilitada por defecto; escritura limitada al workspace. Dos capas: sandbox + política de aprobación.
Prompt injection	La búsqueda web en modo caché reduce el riesgo. Nunca confiar en contenido externo.
Retención de datos	30 días por defecto en API. Zero Data Retention disponible para organizaciones aprobadas.
Secretos en prompts	NUNCA pegar credenciales. Usar variables de entorno. Si se filtran, rotar inmediatamente.
Dependencias	Verificar CVEs, licencias y estado de mantenimiento antes de instalar.
Quality gates	Tests + linters + SAST + scan de dependencias + revisión humana.
Logging seguro	Nunca registrar PII, tokens ni secretos. Usar IDs opacos y hashes.
Definition of Done	El código de IA merece igual o más escrutinio que el código humano.

8. Referencias

- [OpenAI Codex Security Guide](#) — Documentación oficial de seguridad de Codex.
- [OpenAI Enterprise Privacy](#) — Políticas de privacidad y retención para clientes empresariales.
- [OpenAI Business Data](#) — Seguridad, privacidad y cumplimiento para datos empresariales.
- [OpenAI Platform Data Controls](#) — Controles de retención y residencia de datos en la plataforma.
- [OWASP Top 10](#) — Las 10 principales vulnerabilidades web.
- [CWE / MITRE](#) — Catálogo de debilidades comunes en software.